Unit 3

Lesson 3

# Beyond Imagination.

## From Code to Care.

**Time Frame: [90 min]**

### Lesson Objectives:

**By the end of this lesson students will be able to:**

- Learn how to use the Clock component in MIT App Inventor.
- Create a reminder app that helps elderly users remember to drink water and take their medicine.

## Challenge questions of the lesson

- How can we use mobile apps to keep someone safe in real time?
- What happens if a caregiver misses an alert—how can we make that less likely?
- Can Bluetooth communication between devices help improve daily health or independence?
- How would you simplify this app for someone who doesn't use technology often?

## SEL Relation

This lesson encourages students to:

- **Develop empathy:** They are building for someone who may have difficulty seeing, hear-ing, or remembering
- **Communicate responsibly:** By sending alerts that matter, in moments that count
- **Work cooperatively:** App-building requires teamwork, feedback, and shared ideas
- **Feel purpose-driven:** They are not just coding—they are caring through coding
- Students learn designing for others
- They consider real people's needs, such as elders or those who are vulnerable
- Students build confidence through collaborative problem-solving and risk-taking

This lesson connects digital literacy with emotional intelligence — students don't just learn to code, they learn to care through coding."

**Ask students:** "What kind of care would you want if you were the elder or the caregiver? How would your app help?"

# Engage

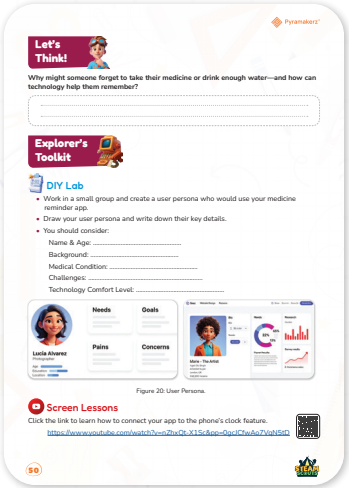
## Teacher's Role:

- Introduce the lesson by guiding students to think about daily routines and why people—especially the elderly or busy individuals—might forget important tasks like taking medicine or staying hydrated.
- Display the question from the student workbook and read it aloud with expression.
- Emphasize that this is a real-life issue affecting many people's health and well-being.
- Ask students to connect the topic to someone they know (a grandparent, parent, or even themselves) who might forget important tasks.
- Prompt deeper thinking by asking:
  - "How do you remember things during your day?"
  - "What tools or reminders do you already use—like phone alarms or apps?"
  - Encourage open discussion and stress that any idea is valid at this stage.

## Expected from Students:

- Reflect on the reasons why someone might forget to take medicine or drink water (e.g., age, busy schedule, memory loss, distractions).
- Think critically about how technology can be used to create reminders or support healthy habits.
- Share personal experiences or observations that relate to the topic.
- Write thoughtful responses in their notebooks, focusing on both the problem and possible tech-based solutions (e.g., mobile alerts, wearable sensors, hydration tracking apps).
- Participate actively in group or partner discussion, building empathy and understanding of the user's needs.

## Parts of the book included.


Let's Think!

Why might someone forget to take their medicine or drink enough water—and how can technology help them remember?

Explorer's Toolkit

DIY Lab

- Work in a small group and create a user persona who would use your medicine reminder app.
- Draw your user persona and write down their key details.
- You should consider:

Name & Age: .....
Background: .....
Medical Condition: .....
Challenges: .....
Technology Comfort Level: .....

Needs | Goals | Pains | Concerns

Figure 20: User Persona.

Screen Lessons

Click the link to learn how to connect your app to the phone's clock feature.

https://www.youtube.com/watch?v=nZhxQk-X1Sc&pp=0gcJCfwAo7VqN5tD

50

# Let's Think!

**Objective:**

Students will identify real-life challenges related to forgetfulness in daily routines and propose technology-based solutions. This reflection encourages empathy, critical thinking, and prepares students to apply design thinking in later stages of the lesson. Students begin to see how simple tech tools can support health, safety, and well-being.

**Answers:**

**Question: Why might someone forget to take their medicine or drink enough water?**

**Sample Answers:**

- They are very busy or distracted during the day.
- They live alone and have no one to remind them.
- They have memory issues, like some elderly people or people with health conditions.
- They don't have a set routine or schedule.

**Question: How can technology help them remember?**

**Sample Answers:**

- Use a mobile app to send daily medication or hydration reminders.
- Set up alarms or voice notifications on a smart device.
- Wear a smartwatch or fitness band that vibrates or shows reminders.
- Create a smart bottle or pillbox that lights up or sends an alert when it's time.

## Possible Strategies for Implementation:

**1. Brainstorm and Share**

Give students 2–3 minutes to think individually and write their ideas. Then, allow pairs to share and compare their answers before opening a brief class discussion. This supports reflection and collaboration.

**2. Use Visual Prompts or Personal Examples**

Show images of pillboxes, water bottles with timers, or screenshots of reminder apps. Ask students:

- "Have you or someone you know used something like this before?"

This makes the problem relatable and builds empathy.

**3. Anchor with a Scenario**

Provide a short user story to ground the thinking:

- "Fatima is 70 and often forgets to take her pills on time. She lives alone. What could help her?"

Ask students to respond from a problem-solver's perspective.

### 4. Categorize Ideas Together

After discussion, list student responses under two columns:

- Challenges people face
- Tech that could help

Use this to scaffold deeper connections and prepare for prototyping or app development.

### 5. UX Mini Discussion

Briefly connect the idea to user-centered design by asking:

- “What would make a reminder tool easy to use for someone older?”
- “How can we make sure it's clear, simple, and helpful?”

## Explorer’s Toolkit

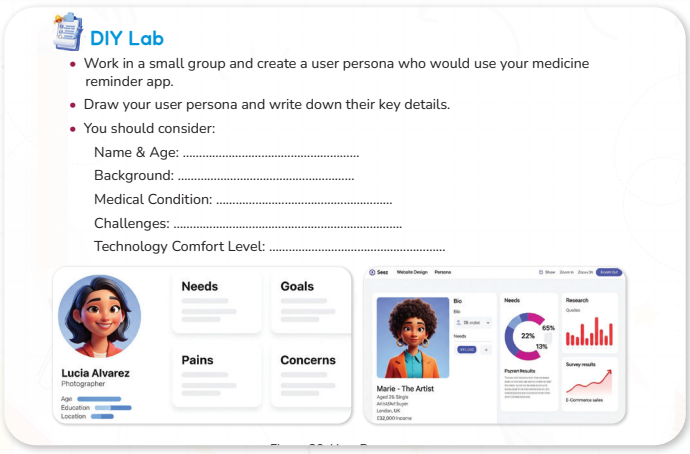
## Teacher's Role:

- **Facilitate Group Work:** Support your team as you create user personas and brainstorm ideas.
- **Guide Design Thinking:** Ask questions to help you think deeply about user needs and crea-tive solutions.
- **Oversee Progress:** Check in on your team's work and provide feedback.

## Expected From Students:

- **Participate Actively:** Engage fully in group discussions and hands-on building.
- **Collaborate:** Work effectively with your teammates to create a user persona.
- **Design Creatively:** Think outside the box to brainstorm useful app features and design a clear user interface.
- **Reflect:** Think about what you learned and how your app could be improved for real users.

## Parts of the book included.



## DIY Lab

### Objective:

Students will work collaboratively to design a realistic user persona who would benefit from a medicine reminder app. By identifying specific needs, challenges, and comfort levels with technology, students will develop empathy for users and lay the foundation for user-centered app design. This activity bridges research and ideation in the engineering design process.

### Materials Needed:

- For User Persona (DIY Lab):
  - Large paper or whiteboard
  - Markers or colored pens
  - Sticky notes (optional)

### Step-by-Step Instructions:

**Design Your User Persona (DIY Lab)**

1. Work in a small group (4-2 students).
2. Create a user persona: Imagine a fictional person who would use your medicine reminder app. This is your "ideal user."
3. Draw your user persona (a simple sketch is fine) and write down their key details on your paper:
   - **Name & Age:** (Give them a specific name and age, like "Grandpa Joe, 75" or "Busy Mom Sarah, 38")
   - **Background:** (Describe their daily life, job, or general routine. E.g., "Retired, loves gardening, lives alone.")
   - **Medical Condition:** (What health issue requires them to take medicine? E.g., "Manages diabetes and high blood pressure.")
   - **Challenges:** (What problems do they have remembering or taking medicine? E.g., "Forgets evening doses," "Has many pills to manage," "Sometimes confuses pills.")
   - **Technology Comfort Level:** (Are they good with smartphones, or do they find apps tricky? E.g., "Uses a basic phone for calls but finds new apps confusing.")

## Possible Strategies for Implementation:

**1. Activate Empathy Through Role Play**

Encourage students to imagine the daily life of someone who struggles with remembering medicine. Ask prompting questions:

- “What does their morning look like?”
- “How might they feel if they forget their medication?”

This helps students move beyond generalizations and create a detailed, human-centered profile.

2. **Provide Persona Examples**

Show examples like “Lucia Alvarez” from the image or fictional users with varied backgrounds (e.g., an elderly man with arthritis, a busy nurse, a teenager managing asthma). This models the depth and format of strong personas.

3. **Group Collaboration with Defined Roles**

Assign roles within each group (e.g., researcher, sketch artist, presenter, writer) to ensure all members contribute meaningfully. Allow flexibility for students to switch roles if needed.

4. **Use Graphic Organizers or Templates**

Offer a pre-structured template with spaces for:

- Name & Age
- Background Story
- Medical Condition
- Challenges
- Technology Comfort Level

This helps scaffold the process and ensures consistency across groups.

5. **Scaffold with Guiding Questions**

Post questions on the board or worksheet:

- “What challenges might this person face?”
- “How do they feel about using apps or smartphones?”
- “What would make a reminder app helpful or frustrating for them?”

6. **Encourage Diversity in Design Thinking**

Challenge students to think beyond stereotypes. Suggest personas from different cultures, age groups, and ability levels. Remind them that good design is inclusive and empathetic.

7. **Share and Reflect**

Once personas are complete, have each group present their user. Ask peers to give warm feedback:

- “What did you like about this persona?”
- “What new idea did it give you for your own design?”

# Explain

## Teacher's Role:

- Introduce the video by explaining that students will now learn how to connect their app to a clock component—an essential feature for scheduling reminders like medicine alerts.
- Emphasize that this is a key part of their app's functionality and must be added with clear logic.
- Encourage students to watch closely and take notes on which blocks are used and how the timer is configured.
- Pause at critical points in the video to clarify terms or recap what's happening in the block editor.

## Expected From Students:

- Watch the video attentively and identify which blocks are used to connect the app to the clock.
- Note down the steps required to set up the clock function for timed reminders or alarms.
- Think about how this timer feature could support their user persona (e.g., how often reminders are needed, what time of day).
- Ask questions during or after the video to ensure clarity and understanding.

## Parts of the book included.

Figure 20: User Persona.

**Screen Lessons**

Click the link to learn how to connect your app to the phone's clock feature.

https://www.youtube.com/watch?v=nZhxQt-X1Sc&pp=0gcJCfwAo7VqN5tD

## Screen Lessons

**Objective:**

Students will learn how to integrate the Clock component in MIT App Inventor to schedule actions (such as sending reminders or triggering notifications). This skill allows students to build time-based functionality into their apps, simulating real-world behavior such as medication alerts or hydration reminders.

Video Link: https://youtu.be/4jctJW2LpzQ?si=yVAO2GP0I3WfXcbI

## Possible Strategies for Implementation:

1. **Pre-Watching**
   - **Set the Context:**
     - Remind students of the goal: "You're designing an app that reminds people to take medicine or drink water. The clock feature is what makes those reminders happen on time."
   - **Preview Vocabulary:** Introduce key terms:
     - Clock component
     - Timer event
     - Interval (milliseconds)
   - **Pose a Focus Question:**
     - Ask: "How does the app know when to do something? What happens when time runs out?"
2. **During Watching**
   - **Watch in Segments:**
     - Pause the video after each major step (e.g., dragging the clock, setting its properties, adding timer logic) and review the purpose of each block.
   - **Guided Notes:** Provide a simple template for students to write:
     - Which blocks are used
     - What each block does
     - What triggers the action (e.g., timer fires every X milliseconds)
   - **Think-Aloud Moments:** Model your thought process while watching:
     - "So, this block fires every time the timer goes off—that's how we remind the user."
3. **After Watching**
   - **Check for Understanding:** Ask students to explain:
     - What the clock component does
     - How it helps the app know when to send a message
   - **Quick Build Challenge:**
     - Have students create a small sample app with a clock that displays a "Time to Drink Water!" message every 10 seconds.
   - **Peer Reflection:** Ask:
     - "How would your persona benefit from this feature?"
     - "What's one way you could personalize the reminder timing?"

# Elaborate

## Teacher's Role:

- Begin this phase by administering the Apprentice assessment to review key vocabulary and core concepts (such as components, variables, and events). This ensures foundational understanding before students begin building.
- Review each question aloud and clarify any misconceptions. Use student responses to identify areas that need quick reteaching or reinforcement.
- Transition into the Builder task by connecting the assessment content to the design challenge:
  - "Now that we know what a variable is and what each component does, let's start applying it to our own app."
- Model how to use the MIT App Inventor interface to place components, referring to the sample design in the student book.
- Encourage students to use their user personas from the Explore phase to guide layout and message tone.
- Provide ongoing support by circulating, offering feedback, and posing guiding UX and coding questions:
  - "What happens when your button is clicked?"
  - "How does your layout support your user's comfort or memory?"

## Expected from Students:

- Complete the Apprentice assessment independently, demonstrating understanding of key interface components, their function, and programming concepts like variables.
- Use feedback from the assessment to reflect on their readiness for app building.
- Apply their knowledge by building a basic screen layout in MIT App Inventor that includes:
  - An image component for the alarm icon
  - Three text input boxes (for time, repetition, and message)
  - A clearly labeled "Set Reminder" button
- Consider their user persona's needs when designing the interface—prioritizing clarity, simplicity, and accessibility.
- Begin basic logic connections using blocks (e.g., setting button click events and storing input as variables).
- Work in pairs or small groups, testing designs, offering peer feedback, and debugging collaboratively.

## Parts of the book included.


Assessment

Apprentice

Multiple-Choice Questions

1. Which App Inventor component would you use to display the 'Alarm Image' on the screen?
   A. Button.
   B. Image.
   C. Canvas.
   D. Label.
2. When a user wants to activate the reminder, they click a component labeled 'Set Reminder'. What type of component is this?
   A. Text box.
   B. Image.
   C. Label.
   D. Button.
3. Which concept is crucial for storing information like the 'Alert Message' or 'No. of times' entered by the user, so it can be used later in the app's logic?
   A. Event
   B. Variable
   C. Properties
   D. Procedures

Builder

You'll Build Your Medicine Reminder App

A. First, Create a basic screen for your medicine reminder app.

It must include the following components:

- Alarm Image
- Text box1 to enter "in each hour"
- Text box2 to enter "No. of times"
- Text box3 to enter "Alert Message"
- Button that says "Set Reminder"

Figure 20: Medicine reminder app interface.

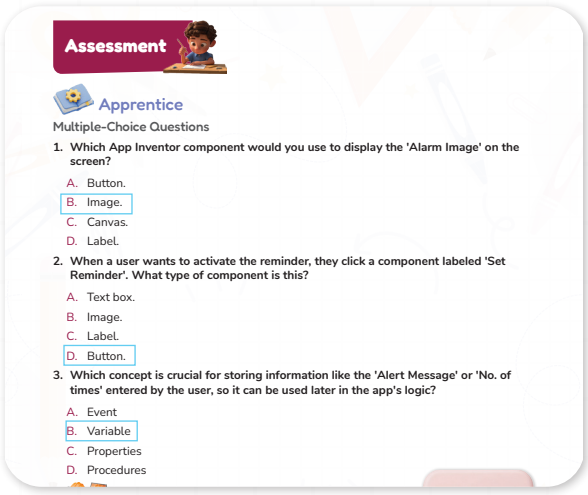
STEAM SCOUTS 51

## Apprentice

### Objective:

Students will demonstrate their understanding of basic MIT App Inventor concepts by answering multiple-choice questions. These include identifying interface components, understanding their function, and recognizing key programming concepts such as variables and user interaction.

### Answers:


Assessment

Apprentice

Multiple-Choice Questions

1. **Which App Inventor component would you use to display the 'Alarm Image' on the screen?**
   A. Button.
   B. Image.
   C. Canvas.
   D. Label.
2. **When a user wants to activate the reminder, they click a component labeled 'Set Reminder'. What type of component is this?**
   A. Text box.
   B. Image.
   C. Label.
   D. Button.
3. **Which concept is crucial for storing information like the 'Alert Message' or 'No. of times' entered by the user, so it can be used later in the app's logic?**
   A. Event
   B. Variable
   C. Properties
   D. Procedures

## Possible Strategies for Implementation:

1. **Pre-Assessment Review**
   - Review visual examples of App Inventor components before the quiz using labeled screenshots or a drag-and-drop demo on the board.
   - Reinforce key vocabulary: Image, Button, Variable, Event, Procedure.
2. **Think–Write–Pair**
   - Have students think through each question and answer it individually.
   - Then, pair up to discuss and justify their answers before final submission.
   - This allows peer teaching and reinforces conceptual understanding.

3. **Active Recall with Live Demo**
   - After the assessment, open App Inventor and point to each component in a real app screen.
   - Ask students to recall what it is and what it does in context.
4. **Error Analysis (Post-Assessment)**
   - Review each question together, asking students to explain why the correct answer works—and why the others don't.
   - Encourage students to reflect on any misunderstandings and write one thing they learned.
5. **Formative Feedback Tool**
   - Use this short quiz not only as an assessment but also as a checkpoint.
   - Use student performance to decide whether more support or reteaching is needed before they begin app building.

## Builder

### Objective:

Students will apply their knowledge of user interface components and user-centered design to create a functional screen layout for a medicine reminder app. Using MIT App Inventor, students will build an interface that includes visual elements (alarm image), input fields (time, frequency, message), and an interactive button—ensuring the layout is clear, accessible, and aligned with their user persona's needs.

### Activity instructions:

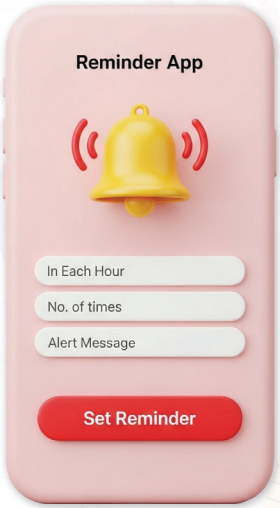
Task: Create a basic user interface for your medicine reminder app in MIT App Inventor.

#### Step 1: Open Your MIT App Inventor Project

- Go to https://ai2a.appinventor.mit.edu/
- Log in with your Google account.
- Click on "Start new project" and name it: MedicineReminderApp

#### Step 2: Set Up the Screen Layout

- In the Designer tab, rename Screen1 to MainScreen (optional but helpful).
- Set the following screen properties:
  - Align Horizontal: Center
  - Align Vertical: Top
  - Background Color: Choose a light or calming color (optional)




#### Step 3: Add the Alarm Image

- In the User Interface drawer, drag in an Image component.
- In the Properties panel, upload an image of a bell or alarm icon (or use the default if none is provided).

- Rename the component to: AlarmImage

### Step 4: Add Input Fields (Text Boxes)

**A. TextBox1: In Each Hour**

- Drag in a Label and set the text to: “In Each Hour”
- Drag in a TextBox underneath it.
- Rename the textbox to: HourInput

**B. TextBox2: No. of Times**

- Drag in a second Label: “No. of Times”
- Add another TextBox
- Rename to: TimesInput

**C. TextBox3: Alert Message**

- Drag in a third Label: “Alert Message”
- Add a third TextBox
- Rename to: MessageInput

**Tips:**

- Set Hint Text in each textbox to guide the user (e.g., “e.g., 2”, “e.g., 3 times”, “Drink Water!”)
- Adjust Width of textboxes to “Fill parent” for better readability.

### Step 5: Add a Button – “Set Reminder”

- Drag in a Button component.
- Set the text to: “Set Reminder”
- Rename the button to: SetReminderButton
- Optional: Customize button color to make it stand out.

### Step 6: Organize Your Layout (Optional Enhancements)

- Use VerticalArrangement to group each label and textbox together for better alignment.
- Add spacing or padding to make the layout clean and readable.

### Step 7: Save and Test Your Layout

- Click “Connect” > “AI Companion” if you want to preview the screen on a mobile device.
- Ensure all components are visible and labeled correctly.
- Ask yourself:
  - Is this clear and easy for my user persona to use?
  - Can I easily read and interact with every part of the screen?

## Possible Strategies for Implementation:

1. **Scaffold the Build with Visual Reference**
   - Display a completed example (like the one shown in the student book image) on the projector or shared screen.
   - Break down the layout visually by component (Image, TextBoxes, Button).
   - Pause after adding each component and confirm students are aligned before moving to the next.
2. **Use a Component Checklist**
   - Provide students with a printed or digital checklist of required UI elements:
     - ☐ Alarm Image
     - ☐ "In Each Hour" Label + TextBox
     - ☐ "No. of Times" Label + TextBox
     - ☐ "Alert Message" Label + TextBox
     - ☐ Set Reminder Button
   - This encourages self-monitoring and supports organization during the build process.
3. **Emphasize UX Design Considerations**
   - Guide students to refer back to their user persona. Ask:
     - "Would this layout be easy for your user to understand?"
     - "Is the text large and clear enough?"
     - "Are buttons and inputs placed where users expect them?"
   - Encourage students to keep designs simple and intuitive.
4. **Build in Phases with Mini-Goals**
   - Split the screen building task into 3 mini-phases:
     - Layout Setup: Background and screen alignment
     - Component Placement: Image and inputs
     - Styling & Adjustments: Width, spacing, text hints, button color
   - Celebrate completion of each phase with quick group check-ins or peer reviews.
5. **Support with Peer Mentoring or Code Buddies**
   - Pair students into "code buddy" teams where they build together or cross-check each other's apps for:
     - Missing components
     - Naming conventions
     - UX consistency

6. **Incorporate Real-Time Teacher Feedback**
   - As students work, circulate and provide verbal feedback framed around:
     - Design clarity (e.g., “Is it clear what each text box is for?”)
     - Usability (e.g., “Would someone elderly know what to press here?”)
7. **Prepare for Logic Integration**
   - Remind students that this is the UI design phase only. Reinforce that logic and functionality will be added in the next phase using the Blocks Editor.
   - This separation helps students focus on interface layout and avoids overwhelm.

## Evaluate

### Teacher's Role:

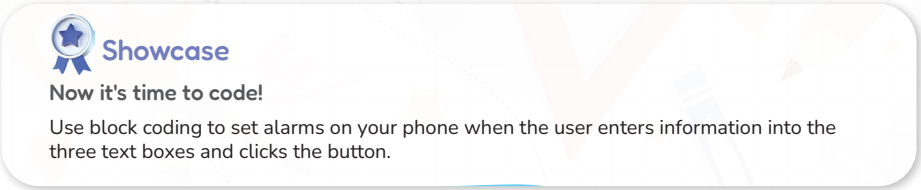
- Introduce the challenge by revisiting the app interface students built in the Builder phase. Explain that now, they will bring the app to life using block-based coding.
- **Clearly outline the goal:** when the user enters data in the three text boxes and clicks the “Set Reminder” button, the app should store the information and activate a timed alert using the clock component.
- Model or review how to:
  - Capture input from text boxes using TextBox.Text blocks
  - Store values in variables
  - Set up a Clock.Timer event or when Button.Click do block to trigger reminders
- Support students by guiding their logic step-by-step, helping them understand when and how blocks are executed.
- Monitor for common errors (e.g., variables not initialized, components not matched properly).
- Facilitate peer review by encouraging students to test each other’s apps and provide feedback on both functionality and user experience.

### Expected from Students:

- Access the Blocks Editor in MIT App Inventor and complete the back-end logic for their reminder app.
- Write functional code that:
  - Stores values from the three input boxes
  - Triggers a timed response using the Clock component
  - Displays the Alert Message on time, based on the user’s input
- Ensure that their code reflects logical structure and UX-friendly behavior (e.g., clear messaging, smooth operation).

- Debug and refine their code through testing and feedback.
- Present their app to peers or the teacher, explaining:
  - How it works
  - What challenges they faced
  - How it helps their user persona

## Parts of the book included.




## Showcase

### Objective:

Students will apply block-based programming in MIT App Inventor to bring their medicine reminder app to life. By coding the app to respond to user inputs (time, frequency, and alert message), they will demonstrate their understanding of variables, user events, and time-based logic. The completed app should activate a functional reminder when the user enters information and clicks the “Set Reminder” button, showcasing both technical accuracy and user-centered design.

### Step-by-Step Instructions:

1. **Open the Blocks Editor**

- Click on the "Blocks" button in the upper right corner.
  Set Up Reminders
- Medicine Reminder Logic:
  - When the "Set Medicine Reminder" button is clicked, capture the time from the medicine Text Box.
  - Use the Clock component to schedule a notification.

### Step 4: Testing Your App

1. **Connect to AI2 Companion**

- Open the MIT AI2 Companion app on your smartphone.
- Scan the QR code displayed on the App Inventor page.

2. **Test Each Feature:** Test setting a medicine reminder and drink water reminder.

- Try your app with test values:
  - Every 2 hours, repeat 3 times, message: “Time to take your medicine!”
- Watch for bugs or logic errors
- If it works, try changing a value or adding a feature
- Reflect: What would make this easier for a real user?

### Step 5: Final Touches

1. **Polish the UI**
   - Customize colors and styles to make the app visually appealing.
   - Ensure all labels, buttons, and notifications are clear and user-friendly.
2. **Adjust Settings**
   - You may want to add settings to modify reminder intervals.

## Possible Strategies for Implementation:

1. **Review Coding Logic as a Class**
   - Begin by revisiting key blocks needed for the task:
     - Get TextBox input (e.g., TextBox.Text)
     - Store data using variables
     - Use Button.Click and Clock.Timer events
   - Model a simplified example on the board or shared screen to clarify how the logic works.
   - Ask guiding questions:
     - "What happens when the button is clicked?"
     - "How do we store and reuse user input?"
2. **Scaffold the Build with Coding Checkpoints**

Break the coding task into smaller, manageable parts. Use checkpoints to help students track progress:

- **Checkpoint 1:** Inputs are stored in variables
- **Checkpoint 2:** Clock is set up and enabled
- **Checkpoint 3:** Alert message is displayed at the correct time

Students check off each part as they test and confirm it works.

3. **Use Peer Debugging Pairs**
   - Partner students to test each other's app functionality.
   - Ask peers to verify:
     - Inputs are being captured
     - The reminder displays accurately
     - The timing matches the user's entry
   - Encourage students to verbalize bugs and explain their code choices to one another.
4. **Reinforce UX Considerations**

Ask students to test their app as their user persona:

- "Would this layout be easy for an elderly user to understand?"
- "Is it obvious what to enter and when to press the button?"

Prompt them to make small design changes if needed to enhance clarity.

5. **Encourage Live Demonstrations**

Once coding is complete, ask students to present their app:

- Explain what it does and how it works
- Describe how it supports the user persona's needs
- Share any challenges they solved during coding

This builds communication skills and reinforces the real-world purpose of the project.

6. **Use a Coding Support Wall**

Display helpful reference blocks or "troubleshooting tips" visibly in the classroom:

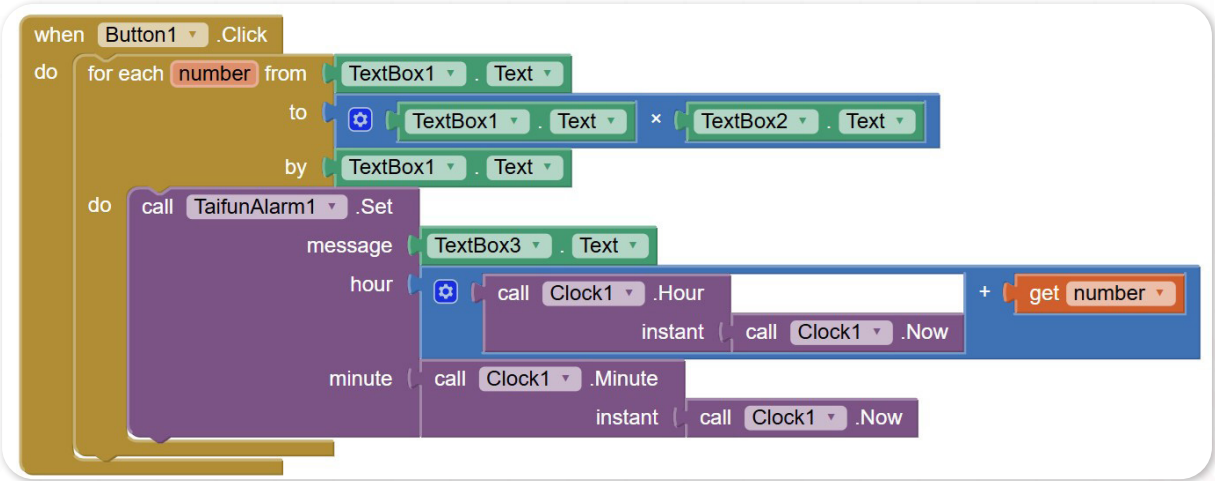
- "Clock not triggering? Make sure it's enabled."
- "Message not showing? Check which label you're setting."

This helps students become more independent problem-solvers.

7. **Celebrate Functionality and Empathy**

Highlight not only technical completion but also empathy-driven design. Recognize apps that:

- Include helpful instructions for the user
- Use large fonts or simple layouts
- Trigger the alert with clear, meaningful feedback




## Possible Strategies for Implementation:

1. **Review Coding Logic as a Class**

- Begin by revisiting key blocks needed for the task:
  - Get TextBox input (e.g., TextBox.Text)
  - Store data using variables
  - Use Button.Click and Clock.Timer events
- Model a simplified example on the board or shared screen to clarify how the logic works.

- Ask guiding questions:
  - "What happens when the button is clicked?"
  - "How do we store and reuse user input?"

**2. Scaffold the Build with Coding Checkpoints**

Break the coding task into smaller, manageable parts. Use checkpoints to help students track progress:

- Checkpoint 1: Inputs are stored in variables
- Checkpoint 2: Clock is set up and enabled
- Checkpoint 3: Alert message is displayed at the correct time

Students check off each part as they test and confirm it works.

**3. Use Peer Debugging Pairs**

- Partner students to test each other's app functionality.
- Ask peers to verify:
  - Inputs are being captured
  - The reminder displays accurately
  - The timing matches the user's entry
- Encourage students to verbalize bugs and explain their code choices to one another.

**4. Reinforce UX Considerations**

Ask students to test their app as their user persona:

- "Would this layout be easy for an elderly user to understand?"
- "Is it obvious what to enter and when to press the button?"

Prompt them to make small design changes if needed to enhance clarity.

**5. Encourage Live Demonstrations**

Once coding is complete, ask students to present their app:

- Explain what it does and how it works
- Describe how it supports the user persona's needs
- Share any challenges they solved during coding

This builds communication skills and reinforces the real-world purpose of the project.

**6. Use a Coding Support Wall**

Display helpful reference blocks or "troubleshooting tips" visibly in the classroom:

- "Clock not triggering? Make sure it's enabled."
- "Message not showing? Check which label you're setting."

This helps students become more independent problem-solvers.

### 7. Celebrate Functionality and Empathy

Highlight not only technical completion but also empathy-driven design. Recognize apps that:

- Include helpful instructions for the user
- Use large fonts or simple layouts
- Trigger the alert with clear, meaningful feedback

## Rubric – Medicine Reminder App (10 points)

| Criteria | Excellent (2 pts) | Good (1 pt) | Needs Work (0 pts) |
|---|---|---|---|
| UI Design | Clear, clean, accessible layout with renamed components | Mostly clear with minor issues | Confusing or incomplete layout |
| Code Logic | Blocks correctly set timer and message | Minor errors, mostly functional | Major errors, logic not working |
| Component Use | Uses all 3 TextBoxes, button, Notifier, and Clock | Missing 1 component or misused | Several components missing or unclear |
| Testing & Iteration | App tested and improved at least once | Basic test, no improvements | Not tested |
| Empathy & Purpose | Clearly designed with user needs in mind | Some thought to user experience | No real-world purpose explained |

**Don't forget to go back and add this to improve part.**

## Now I Can

**Objective:**

Students will reflect on their learning and identify how they've grown in understanding communication, empathy, and the role of assistive tools in helping others share ideas.

**Activity Instructions:**

1. **Introduce the Reflection**
   - Say: "Let's look back on everything we've explored this lesson. You learned how it feels to face communication challenges, and how tools can help people share their thoughts in special ways."
2. **Guide Students Through the Checklist**
   - Read each statement aloud one at a time.
   - After each one, ask students to give a thumbs up, sideways, or down to show how confident they feel.
   - Example prompts:
     - "Did you learn how people share ideas without talking?"
     - "Do you know how tools can help people talk?"
3. **Student Completion**
   - Let students check off or color the "Now I Can" boxes they feel good about.
   - Remind them: "It's okay if you didn't check all of them. We're all still learning."
4. **Optional Extension**
   - Ask students to circle the one they're most proud of.
   - Invite them to write a sentence or draw a picture to show their favorite learning moment.

## Relation to Standards

1. **NGSS – Next Generation Science Standards (Middle School)**
   - MS-ETS1-1: Define a real-world health problem (missed medicine reminders) and identify criteria for a successful app-based solution.
   - MS-ETS1-2: Evaluate different interface and logic designs to determine which best supports the user's needs.
   - MS-ETS1-3: Build and improve a functional prototype of a medicine reminder app using testing and user feedback.
2. **ISTE Standards for Students (International Society for Technology in Education)**
   - 1. Empowered Learner: Students design an app that addresses a personal health and wellness challenge, taking ownership of their learning.

- 4. Innovative Designer: Students apply the design process to create, test, and refine an app using MIT App Inventor.
- 5. Computational Thinker: Students use sequencing, variables, and logic in block-based coding to manage time-based app functions.
- 6. Creative Communicator: Students present ideas through interface design and reflection, explaining how their app benefits others.

### 3. UN Sustainable Development Goals (SDGs)

- SDG 3 – Good Health and Well-Being: Students build a reminder tool that supports users in managing daily health routines and medication schedules.
- SDG 4 – Quality Education: Students participate in inclusive, hands-on STEAM learning using technology and real-world challenges.

### 4. CSTA Computer Science Standards

- Algorithms & Programming: Students use block-based code to process input and trigger reminders through event-based logic.
- Computing Systems: Students understand how input (text boxes), processing (variables), and output (alerts) work together in their app.
- Human-Computer Interaction: Students design with empathy, making sure their apps are accessible and easy to use for diverse users.
- Testing & Debugging: Students test and revise their app's logic to ensure it performs reliably and supports its purpose.

### 5. 21st Century Skills (P21 Framework)

- Critical Thinking: Students identify problems in their code and apply logical thinking to solve them.
- Collaboration: Students work in pairs to build, test, and give feedback on their app designs.
- Communication: Students explain their app's function and user-centered features through presentations and peer discussions.
- Creativity: Students personalize their app layout and features to reflect thoughtful, real-world solutions.

# Mission Innovate

## Final Mission – Design for Connection and Companionship

### Overview:

In this culminating, open-ended project, students will design a tech-based solution to help elderly people feel more connected, supported, and emotionally well—especially if they spend more time alone or have difficulty joining social activities. The goal is to create a tool, device, or system that helps older adults stay in touch with family, friends, or caregivers and feel less isolated.

Project duration: 2 weeks

## Teacher's Role:

**Facilitator of Inquiry:**

- Begin by reading the mission prompt aloud. Spark curiosity by asking:
  - "What makes older people feel lonely or disconnected?"
  - "What could we design to help them feel cared for and connected?"
- Support students as they brainstorm emotional and social challenges faced by elderly people.

**Encourager of Empathy and Design Thinking:**

- Guide students to research existing tools that promote emotional well-being, such as video calls, reminder notes, or voice assistants.
- Encourage them to think about physical, emotional, and cognitive needs when planning solutions.

**Supporter of Student-Led Innovation:**

- Allow students to choose a problem to solve, sketch or write their ideas, and decide on the features their design should include.
- Provide flexibility in prototyping—using recycled materials, digital tools like MIT App Inventor, mBlock, Arduino, or PictoBlox.

**Guide for Reflection:**

- Prompt students to reflect on:
  - "How does your design make older people feel emotionally connected?"
  - "What specific features in your design address loneliness?"
- Encourage peer testing by simulating real scenarios and gathering feedback.

## Assessment Guide:

**Use a rubric to evaluate:**

- Clear identification of the problem and target user needs
- Creativity and feasibility of the solution
- Functionality and relevance of prototype features

- Clarity and empathy in presentation of the idea
- Collaboration and problem-solving effort

## Implementation Tips:

- Allow students to work individually or in teams for idea sharing and peer support.
- Provide access to:
  - Craft materials (cardboard, paper, LEDs, buzzers)
  - Computers or tablets with coding/design software
  - Internet access for researching assistive technologies
- Encourage students to test their design with a peer acting as the “end user” to evaluate usability and emotional impact.
- Dedicate time for students to present via gallery walk, video pitch, or live demonstration.